CASE STUDY · N°03 · PARIS

SantéFlux

Building the Platform

THE TWO FILES

`patients` · one row per person

`events` · one row per episode

By Friday: one table, versioned, and a job that rebuilds it.

Two files, and a platform to build from them. A Paris health service holds a table of patients and a table of clinical episodes, and a team that still answers every question by hand. The board wants reporting the current setup cannot safely produce. **Reusable, versioned, scheduled** is the mandate. The build is the answer.

SIX SECTIONS

I Shared functions

II Join & enrich

III Version the table

IV Schedule the job

01 · EXECUTIVE BRIEF

Stop answering by hand.

SantéFlux must move from ad-hoc, hand-run notebooks to one automated, versioned platform. Two extracts have landed in the workspace, and the board now wants regional reporting the current setup cannot safely produce.

Situation. Two files: `patients`, one row per person, and `events`, one row per clinical episode. Care providers want regional trend and cost figures drawn from both together.

Complication. The derivation logic is copied between notebooks and has already drifted. The reporting table is overwritten every morning, so a figure that looks wrong on Thursday cannot be compared with yesterday, explained, or rolled back. And nothing runs unless an engineer is at the keyboard pressing cells.

Resolution. One reusable platform: the derived fields written as shared functions, the two tables joined and enriched, the result a versioned Delta table that can be inspected and rolled back, and a scheduled job that rebuilds it unattended.

It is a question of discipline, not of features. Build it so the next engineer reruns it without asking how, and so last week is never lost to this morning's write.

IN ÉLISE'S WORDS

> **“If a number looks wrong on Thursday, I need to see what changed and roll it back. Right now I cannot even tell you what yesterday's table held.”**

Élise Fontaine · Data Lead, SantéFlux

CLIENT OVERVIEW

A reporting team, becoming a platform.

From notebooks to a platform. SantéFlux aggregates hospital and pharmacy records into analytics for care providers across France. Until now each report has been a hand-run notebook, assembled the morning it is needed.

The board mandate. Providers want dependable regional figures, and the same numbers reproduced on demand. The current path can promise neither: it forgets its own history and depends on a person to run it. The team must mature from ad-hoc scripts to a scheduled, versioned platform.

3×
the same derivation logic, copied across notebooks and already drifting apart.

0
versions kept: each morning's write overwrites the last, nothing to roll back to.

72h
a week of mornings, each needing an engineer at the keyboard to produce the report.

CURRENT PRACTICE, AND THE TARGET

KEY METRIC	CURRENT PRACTICE	TARGET PLATFORM
Derivation logic	Copied into each notebook	Shared functions, defined once
Reporting table	Overwritten every morning	Versioned Delta, travel and restore
Execution	Manual, cell by cell	Scheduled job, unattended

02 · PROBLEM DIAGNOSIS

Three failures. One fragile report.

A review of how the morning report is produced finds a break at every point where the work should have been made reusable, durable, or automatic.

01 Copied logic, drifting

The age band, the cost flag and the risk score are written inline, pasted into whichever notebook needs them. Two copies already disagree, so the same patient scores differently in two reports.

02 Overwritten every morning

The reporting table is rebuilt with a full overwrite each day. There is no history, so a figure that looks wrong cannot be compared with yesterday, explained, or rolled back. The evidence is gone the moment it is replaced.

03 Runs only by hand

Every report exists because someone pressed the cells in order. Nothing is parameterised, nothing is scheduled. A day off, and the providers get no numbers.

THE PATTERN

Each failure is the absence of one discipline: reuse, versioning, automation. The platform is what supplies all three, in that order.

03 · THE PLATFORM DESIGN

Shared, joined, versioned, scheduled.

One path from the two raw files to a report the team can trust and reproduce. Four moves, each fixing one of the three failures.

Move I

Shared functions

The derived fields become named functions in a notebook of their own, so the logic lives once and every notebook draws on the same copy.

Move II

Join & enrich

The two tables are joined on the patient key and enriched with the derived fields, producing one analytic row per episode.

Move III

Version the table

The result is written as Delta. Every write is a version, so the table can be read as it was, and a bad change rolled back.

Move IV

Schedule the job

The main notebook is parameterised and wired to a job, so the whole platform rebuilds on a timer with no one at the keyboard.

DESIGN PRINCIPLE

Reuse, then history, then automation.

Write the logic once and share it; build the table so it remembers; make the run belong to a schedule, not a person. The order is the engineering discipline this case exists to teach. Reverse it, and the platform ships the very fragility it was built to remove.

04 · ENGINEERING MANDATE

You are the engineer.

Build two notebooks. One holds only the shared functions; the other loads the data, pulls those functions in, joins, enriches, versions and schedules. The code you write decides whether the next engineer can rerun it.

NON-NEGOTIABLE

The functions live in their own notebook, defined once. The reporting table is never a blind overwrite that loses its history. The scheduled run takes the region as a parameter, not a value typed into a cell.

EXHIBIT 1 · events JOINED TO patients ON patient_id

Both tables carry a region; the join must not let the two collide.

PATIENT_ID	DATE_OF_BIRTH	DIAGNOSIS	REGION	EVENT_TYPE	COST_EUR
P-00412	1984-03-11	E11.9	Paris	consultation	48.00
P-01907	1958-08-02	I10	Lyon	laboratoire	82.50
P-00088	2010-12-19	J45	Marseille	prescription	21.00

THREE FIELDS TO DERIVE

An age band, a cost flag, and a risk score, drawn from these rows. The next page states the rule behind each. The implementation is yours to write.

DERIVATION CONTRACT

The rule, not the code.

Every derived field has a rule. Write each as a reusable function that takes a column and returns a column. The third is built from the first two.

FIELD	TYPE	DERIVATION RULE	CONSTRAINT
`age_band`	String	Banded from date of birth: under 18, 18 to 39, 40 to 64, 65 and over.	*Reusable function*
`cost_flag`	String	“high” when the episode costs fifty euro or more, otherwise “normal”.	*Reusable function*
`risk_score`	Integer	Two points if the age band is 65 and over, two if the diagnosis is a chronic code (diabetes or hypertension), one if the cost flag is high. Summed.	*Built from the other two*

WHY FUNCTIONS, AND WHY SHARED

`risk_score` reuses `age_band` and `cost_flag`: that is the reason to name them rather than inline them. All three sit in one notebook the main notebook pulls in, so a corrected rule lands everywhere at once instead of in one copy of three.

05 · ACCEPTANCE CRITERIA

Four gates. Pass all, or it is not a platform.

The build is judged against what Élise promised the board. Any gate failed, and it is back to hand-run scripts.

01
Reuse

The three derived fields are defined once, as shared functions, and pulled into the main notebook rather than copied. A rule fixed in one place takes effect everywhere.

02
Reproducibility

The join carries no ambiguous region column, and the enriched table is written as Delta so any run can be reproduced and inspected later.

03
Recoverability

A change to the table leaves a version behind. It can be read as it was, and a bad change rolled back, without restoring from a backup.

04
Unattended run

The region is a parameter, and a scheduled job rebuilds the whole platform for a given region with no one at the keyboard.

VALIDATION

The enriched table carries age_band, cost_flag and risk_score. Its history shows more than one version. A change to one region can be read before and after, then restored. The job runs for a passed region with no manual step.

06 · DISCUSSION

Six questions. Reasoning, not recall.

Q1 · Reuse vs speed. Why define the three derived fields as shared functions rather than inline them where each is needed? What does inlining cost the morning the rule changes?

Q2 · Sharing across notebooks. How does the main notebook make the functions notebook's definitions available without rerunning its work, and why does the path between them matter?

Q3 · The join. Both tables carry a region. What goes wrong on a naive join, and what sequence keeps it unambiguous? When is it right to pull values back to Python, and when is it wrong?

Q4 · Why versions. A figure looks wrong on Thursday. What does a versioned Delta table let you do that a nightly overwrite does not? Name the read, and the rollback.

Q5 · The cost of cleanup. Reclaiming old files frees storage but ends time travel to versions that needed them. How do you reason about when to run it, given Élise may want to inspect last week?

Q6 · Making it unattended. The region is hard-coded in a cell. What turns the notebook into something a schedule can drive for any region, and how does a job hand it that value?

LAB INSTRUCTIONS · YOU HAVE UNTIL FRIDAY

Two notebooks. Functions, then platform.

Build in order. One notebook holds only the shared functions. The main notebook pulls them in, joins, enriches, versions the table, and is the one a job runs on a schedule. Bring working code and the history and version evidence, not prose.

NOTEBOOK	WHAT IT MUST DO	PRODUCES
Functions	Define age_band, cost_flag and risk_score as column-in, column-out functions. Nothing else: no loading, no work.	Three shared functions
Main	Take a region parameter, load both tables, pull in the functions, join without a region collision, enrich, then write the table as Delta. Update it, read its history, travel to an earlier version, restore, and clean up. Finally, schedule it as a job.	Versioned table + job

REMINDER

Reuse first, history second, automation last.

Functions shared before the join, the table versioned before it is trusted, the run handed to a schedule before the week depends on it. The order is the engineering discipline this case exists to teach. The platform is the answer; the report is what it produces every morning without you.

From two files to a platform.

By Friday, Camille can hand Élise a platform, not a pile of cells. The derived fields written as functions and shared across notebooks, the two tables joined and enriched, the result a Delta table with a history she can travel and restore, and a job that rebuilds it on a schedule. The next engineer can rerun it without asking how.

I Shared functions.

II Join and enrich.

III Version the table.

IV Schedule the job.